

IHK Lernheft 2025 – Entwicklung & Umsetzung von Algorithmen (FIAE)

Java-ähnlicher Pseudocode (ohne Datentypen) · druckfreundlich · 1 Thema = 1 Abschnitt

Zählen, Summieren, Durchschnitt

Pseudocode

```
funktion zaehleGroesser(a, grenze)
  count = 0
  for (i = 0; i < a.length; i++)
    if (a[i] > grenze) count++
  return count
```

```
funktion durchschnitt(a)
  sum = 0
  for (i = 0; i < a.length; i++) sum = sum + a[i]
  return sum / a.length
```

Beispiel

$a = \{5, 7, 2, 9\}$, $grenze = 6 \rightarrow count = 2$; Durchschnitt = 5.75

Tipp: Zählen = $count++$; Durchschnitt = sum / n .

Min / Max (1D)

```
funktion findeMin(a)
  min = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] < min) min = a[i]
  return min
```

```
funktion findeMax(a)
  max = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] > max) max = a[i]
  return max
```

$a = \{5, 7, 2, 9\} \rightarrow \min = 2, \max = 9$

Startwert = $a[0]$, nicht 0.

Min / Max (2D)

```
funktion findeMin2D(m)
  min = m[0][0]
  for (i = 0; i < m.length; i++)
    for (j = 0; j < m[i].length; j++)
      if (m[i][j] < min) min = m[i][j]
  return min
```

```
funktion ermittleJahresstatistik(verbrauch) // Monat×Tag
  min = MAX_WERT
  max = MIN_WERT
  for (m = 0; m < verbrauch.length; m++)
    summe = 0
    for (t = 0; t < verbrauch[m].length; t++) summe = summe + verbrauch[m][t]
    if (summe < min) min = summe
    if (summe > max) max = summe
  return new Jahresstatistik(min, max)
```

2D-Aufgaben = verschachtelte Schleifen + Vergleich/Akkumulation.

Objektrückgabe (Jahresstatistik)

```
klasse Jahresstatistik(min, max)
  // Attribute: minVerbrauch, maxVerbrauch
  // Konstruktor setzt Werte
  // Getter: getMinVerbrauch(), getMaxVerbrauch()
```

```
funktion ermittleJahresstatistik(a)
  // ... min/max berechnen ...
  return new Jahresstatistik(min, max)
```

 In der Prüfung explizit new Klassenname(...) schreiben.

If/Else + Counter (z. B. „besser als DAX“)

```
funktion notierungPlus(kurse)
  count = 0
  for (i = 0; i < kurse.length; i++)
    if (kurse[i].aktieProzent > kurse[i].daxProzent) count++
  return count
```

Immer count++ im wahren Zweig; sonst am Ende return count.

BubbleSort

```
funktion bubbleSort(a)
  for (i = 0; i < a.length - 1; i++)
    for (j = 0; j < a.length - 1 - i; j++)
      if (a[j] > a[j+1]) swap(a[j], a[j+1])
```

Erkennbar an „benachbarte Elemente vertauschen“; innere Grenze: $n-1-i$.

InsertionSort

```
funktion insertionSort(a)
  for (i = 1; i < a.length; i++)
    key = a[i]
    j = i - 1
    while (j >= 0 && a[j] > key)
      a[j+1] = a[j]
      j = j - 1
    a[j+1] = key
```

 Gut bei fast sortierten Arrays; stabil.

● Lineare Suche

```
funktion linearSearch(a, x)
  for (i = 0; i < a.length; i++)
    if (a[i] == x) return i
  return -1
```

Sobald gefunden → return; sonst am Ende -1.

● Objektvergleich mit Function vergleiche

```
funktion sort(a, vergleiche)
  for (i = 0; i < a.length - 1; i++)
    for (j = 0; j < a.length - 1 - i; j++)
      if (vergleiche(a[j], a[j+1]) > 0) swap(a[j], a[j+1])
```

vergleiche(x,y) benutzen – nicht direkt $x > y$ bei Objekten.

● BinarySearch (selten)

```
funktion binarySearch(a, x)
  left = 0; right = a.length - 1
  while (left <= right)
    mid = (left + right) / 2
    if (a[mid] == x) return mid
    if (a[mid] < x) left = mid + 1
    else right = mid - 1
  return -1
```

Nur bei sortierten Arrays; in AP2 selten.

● Rekursion (Fibonacci)

```
funktion fib(n)
  if (n <= 1) return 1
  else return fib(n-1) + fib(n-2)
```

(Ineffizient; iterativ viel schneller. Eher Theorie.

● QuickSort / MergeSort (Theorie)

Grob kennen reicht: QuickSort (Pivot, partitionieren), MergeSort (teilen & zusammenführen). Ø $O(n \log n)$.